Learning the Cost of Rerouting: Progress Report 2

EE-353 Computer Networks. Work after the Stage 2 milestone: where does a reroute really hurt? (14 November 2026)

Name	CMS ID	Section
Mutahar	540395	BESE-16 A
Uswa Fatima	575207	BESE-16 A

Source code and all raw results: https://github.com/MuhammadMuthar/sdn-reroute, branch *claude/chunk7-regime*

1. Why this step

The Stage 2 report ended with an honest problem. Rerouting big flows beats ECMP by 2.59 Mbit/s per flow, but on our 10 Mbit/s network one reroute costs almost nothing, so our damage predictor had nothing to predict and our controller only matched the Hedera-style baseline. The proposal (Section V-A) had also promised a pilot under TCP Cubic and BBR, each with RACK loss detection on and off, which was still open. This step does both: it runs that pilot and searches for the conditions where a reroute really costs throughput.

2. What we ran

One overnight campaign on the laptop (11 h 45 min, 13 to 14 November). We varied four things: link rate (10 and 100 Mbit/s), congestion control (Cubic and BBR), RACK (on and off) and the delay of the slowest path (10 ms as before, or 50 ms; the fastest path is 1 ms). That gives 16 settings. In each setting we tested four kinds of move: to a faster path or to a slower path, and onto an empty path or onto a path that already carries another big flow. Each kind was run as 5 matched pairs: the same traffic once with the move, and once with the flow on the destination path from the start. The damage of a move is the goodput the destination path loses against that matched run, in the 10 s after the move. We also moved one flow back and forth every 0.5 s and every 2 s for 10 s, because an eager controller does that. In total 640 runs (320 matched pairs), 30 back-and-forth runs and 16 check runs. All ran without error and none had to be excluded. Every run recorded that the TCP settings were really in effect on all 8 hosts. The plan and the rule for calling something damage (confidence interval above zero and at least 1 % of the link) were written into the lab log before the first run.

3. Results

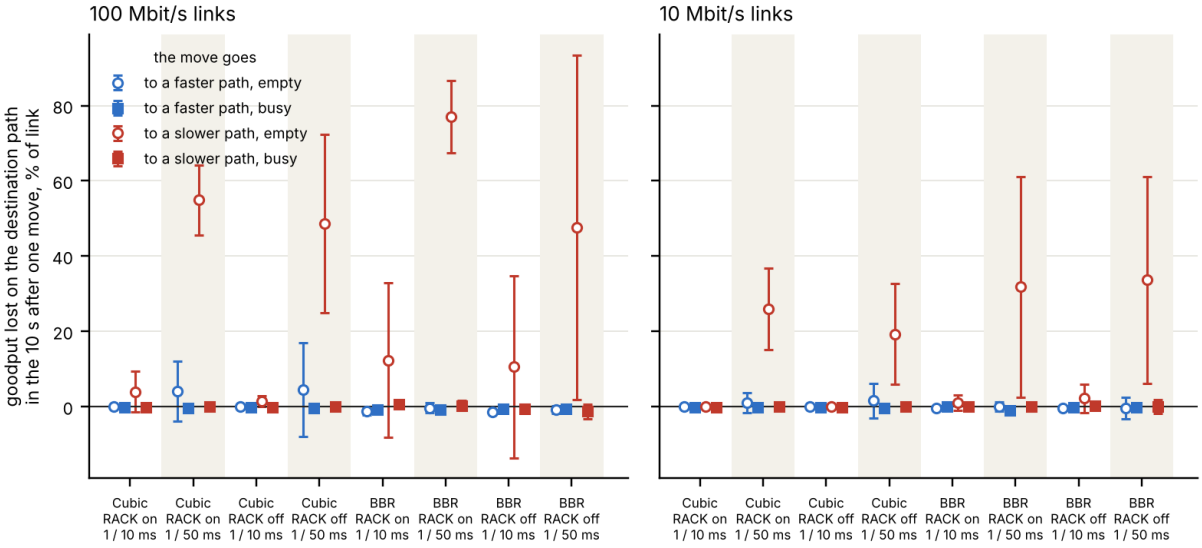


Figure 1. Goodput one reroute costs the destination path, mean and 95 % confidence interval over 5 matched pairs per point. Shaded columns are the settings with the 50 ms slow path. Only one kind of move stands out: onto a slower, empty path.

Move onto a slower, empty path; goodput lost in 10 s, % of link (mean ± 95 % CI, n = 5)	Cubic, RACK on	Cubic, RACK off	BBR, RACK on	BBR, RACK off
100 Mbit/s, slow path 50 ms	**54.9 ± 9.3**	**48.7 ± 23.7**	**77.1 ± 9.6**	**47.6 ± 45.8**
10 Mbit/s, slow path 50 ms	**25.9 ± 10.8**	**19.3 ± 13.4**	**31.8 ± 29.2**	**33.7 ± 27.5**
100 Mbit/s, slow path 10 ms	4.0 ± 5.5	1.4 ± 1.4	12.3 ± 20.5	10.6 ± 24.3
10 Mbit/s, slow path 10 ms	0.05 ± 0.20	0.09 ± 0.28	1.0 ± 2.1	2.2 ± 3.8

Table 1. The one kind of move that costs something. Bold cells meet the damage rule. The other 56 of the 64 tested cases do not.

Reordering is not the cost. Moves to a faster path reorder packets heavily (up to 325 reordering events and 555 retransmissions in 2 s, as averages per case), yet the path loses nothing measurable, in every setting. Switching RACK off did not change this, and BBR was not gentler than Cubic. Moving a flow back and forth every 0.5 s cost it less than 1 % of its goodput (it kept 99.6 % at 100 Mbit/s and 99.2 % at 10 Mbit/s). Three things we expected to hurt therefore did not, and we report that as measured.

The cost is arriving on a long, empty path with a sending window sized for the short one. When a flow moves from the 1 ms path to an empty 50 ms path, its window is far too small for the new path and takes many seconds to grow (Cubic), or its delay estimate is stale (BBR). The path then carries 19 to 77 % less than it would have, for 10 s. At 100 Mbit/s with Cubic that is 68.6 MB lost by a single move. With the 10 ms slow path the same move shows a smaller effect that does not meet our rule (4.0 ± 5.5 % at 100 Mbit/s).

A busy destination hides the cost. Onto a path that already carries a big flow, the same move costs the path nothing (+0.10 ± 0.34 % at 100 Mbit/s, Cubic, 50 ms). The moved flow itself still loses (about 36 % of the link in that case), but the flow already there takes what it leaves.

Check. We recomputed the table a second way, from iperf3's own receiver reports instead of packet traces, with separate code, for the 12 of these 16 cases whose raw runs are uploaded so far. The same 6 cases show damage and the largest difference between the two methods is 2.0 percentage points.

4. What this means for our improvement

This is the first setting where the proposal's idea has something to work with. A Hedera-style controller always picks the least-loaded path, and the least-loaded path is exactly the empty one. If that path is much longer, the move throws away up to half of the link for several seconds, and the controller never sees why. The damage also depends on things the controller already knows before it moves a flow: how much longer the new path is, whether it is empty, and how fast the flow is sending. So a cost estimate can tell a harmless move from a costly one, which was not possible on the first network where every move was harmless.

Two limits, stated plainly. First, this damage is not the reordering damage the proposal expected; it is a ramp-up cost, and the paper will say so. Second, a 1 ms against 50 ms path difference is typical of a wide-area network, not of a data-centre fabric. With data-centre-like differences (1 to 10 ms) we still find no damage.

5. Next steps

(1) Run the five-controller comparison (ECMP, Hedera-style, cooldown-only, cost-aware with formula, cost-aware with ML) in the setting where damage exists: 100 Mbit/s, paths of very different length, traffic in which the least-loaded path is often the long empty one. Add three inputs to the cost model: the delay ratio of new to old path, the flow's window against what the new path needs, and whether the new path is empty. (2) Build the k=4 fat-tree and the AI-training traffic from the proposal. (3) Train on one network and test on another. Not started yet: the comparison in the new setting, the fat-tree, the AI-training traffic and the margin-only ablation.